

Document number: P0457R2
Date: 2017-11-11
Project: LWG
Reply-to: Mikhail Maltsev <maltsevm@gmail.com>

String Prefix and Suffix Checking

1. [Abstract](#)
2. [History](#)
 1. [Changes from R0](#)
 2. [Changes from R1](#)
3. [Motivation](#)
4. [Prior work](#)
5. [Design considerations](#)
 1. [Member function vs free function](#)
 2. [Overload set](#)
 3. [Passing by value vs passing by reference](#)
6. [Wording](#)
 1. [basic_string](#)
 2. [basic_string_view](#)
7. [References](#)

1. Abstract

This paper proposes to add member functions `starts_with` and `ends_with` to class templates `basic_string` and `basic_string_view`. These functions check, whether or not a string starts with a given prefix or ends with a given suffix, respectively.

2. History

2.1. Changes from R0

- Overloads accepting `const charT*` were added
- Behaviour of all operations (except the two mentioned below) was expressed in terms of the following two methods of the `basic_string_view` template:
 - `constexpr bool starts_with(basic_string_view<charT, traits> s) const noexcept;`
 - `constexpr bool ends_with(basic_string_view<charT, traits> s) const noexcept;`

2.1. Changes from R1

- Dropped `noexcept` for `const charT*` overloads
- More concise wording

3. Motivation

Checking, whether or not a given string starts with a given prefix (or ends with a given suffix) is a common task. In fact, standard libraries of many other programming languages include routines for performing such checks, for example:

- Python: `str` and `unicode` classes have `startswith` and `endswith` methods [1].
- Java: class `String` has `startsWith` and `endsWith` methods [2].

And so on.

Also, some C++ libraries (other than the standard library) that implement a string type include such methods. For example, Qt library has classes `QString` [3] and `QStringRef` (analogous to `std::string_view`) which have `startsWith` and `endsWith` member functions.

These functions are widely used. For example, the source code of a recent version of Qt (excluding third-party components) has 1193 occurrences of `startsWith` and 953 occurrences of `endsWith`. Other examples include Webkit (304 occurrences of `startsWith` and 142 occurrences of `endsWith`) and LLVM (class `StringRef`, 113 matches for `StartsWith` and 38 matches for `EndsWith`).

4. Prior work

The `basic_string_view` class template included `starts_with` and `ends_with` up to revision 3 of the proposal (N3609 [4]). These two member functions were removed after LEWG discussion in Bristol. The main concerns were:

- `basic_string_view` should have an API consistent with `basic_string`;
- changes to `basic_string` should go to a separate proposal.

5. Design considerations

5.1. Member function vs free function

This proposal adds member functions `starts_with` and `ends_with` to class templates `basic_string` and `basic_string_view`. Another considered option was to add free functions to namespace `std`, but adding member functions is consistent with the existing API for `compare`. Besides, as the original proposal [5] mentioned, the order of parameters of a free function is ambiguous (`starts_with(string, prefix)` VS `starts_with(prefix, string)`).

5.2. Overload set

The original `string_view` proposal included the following overloads of `starts_with`:

```
// basic_string:
bool starts_with(basic_string_view<charT, traits> s) const noexcept;
bool starts_with(charT c) const noexcept;
bool starts_with(const charT* s) const noexcept;

// basic_string_view:
bool starts_with(const basic_string_view& prefix) const noexcept;
```

Qt offers the following overload set (for `QString` and `QStringRef`):

```
bool startsWith(const QString & s, Qt::CaseSensitivity cs = Qt::CaseSensitive) const
bool startsWith(const QLatin1String & s, Qt::CaseSensitivity cs = Qt::CaseSensitive) const
bool startsWith(const QChar & c, Qt::CaseSensitivity cs = Qt::CaseSensitive) const
bool startsWith(const QStringRef & s, Qt::CaseSensitivity cs = Qt::CaseSensitive) const
```

This proposal includes the following overloads:

```
// basic_string:
bool starts_with(charT x) const noexcept;
bool starts_with(basic_string_view<charT, traits> x) const noexcept;
bool starts_with(const charT* x) const;

// basic_string_view:
constexpr bool starts_with(charT x) const noexcept;
constexpr bool starts_with(basic_string_view<charT, traits> x) const noexcept;
constexpr bool starts_with(const charT* x) const;
```

The overload overload for `basic_string` is not included, because `basic_string` has a non-explicit conversion operator to `basic_string_view`.

5.3. Passing by value vs passing by reference

In accordance with the guidance from P0254R1 [5], objects of type `basic_string_view` are passed by value (not by reference).

6. Wording

6.1. `basic_string`

In `[basic.string]`, add:

```
bool starts_with(basic_string_view<charT, traits> x) const noexcept;
bool starts_with(charT x) const noexcept;
bool starts_with(const charT* x) const;
bool ends_with(basic_string_view<charT, traits> x) const noexcept;
bool ends_with(charT x) const noexcept;
bool ends_with(const charT* x) const;
```

After `[string::compare]` add:

basic_string::starts_with [string.starts_with]

```
bool starts_with(basic_string_view<charT, traits> x) const noexcept;  
bool starts_with(charT x) const noexcept;  
bool starts_with(const charT* x) const;
```

Effects: Equivalent to: `return basic_string_view<charT, traits>(data(), size()).starts_with(x);`

basic_string::ends_with [string.ends_with]

```
bool ends_with(basic_string_view<charT, traits> x) const noexcept;  
bool ends_with(charT x) const noexcept;  
bool ends_with(const charT* x) const;
```

Effects: Equivalent to: `return basic_string_view<charT, traits>(data(), size()).ends_with(x);`

6.2. basic_string_view

In [string.view.template], add:

```
constexpr bool starts_with(basic_string_view x) const noexcept;  
constexpr bool starts_with(charT x) const noexcept;  
constexpr bool starts_with(const charT* x) const;  
constexpr bool ends_with(basic_string_view x) const noexcept;  
constexpr bool ends_with(charT x) const noexcept;  
constexpr bool ends_with(const charT* x) const;
```

In [string.view.ops], add:

```
constexpr bool starts_with(basic_string_view x) const noexcept;
```

Effects: Equivalent to: `return compare(0, npos, x) == 0;`

```
constexpr bool starts_with(charT x) const noexcept;
```

Effects: Equivalent to: `return starts_with(basic_string_view(&x, 1));`

```
constexpr bool starts_with(const charT* x) const;
```

Effects: Equivalent to: `return starts_with(basic_string_view(x));`

```
constexpr bool ends_with(basic_string_view x) const noexcept;
```

Effects: Equivalent to: `return size() >= x.size() && compare(size() - x.size(), npos, x) == 0;`

```
constexpr bool ends_with(charT x) const noexcept;
```

Effects: Equivalent to: `return ends_with(basic_string_view(&x, 1));`

```
constexpr bool ends_with(const charT* x) const;
```

Effects: Equivalent to: `return ends_with(basic_string_view(x));`

7. References

1. The Python Standard Library. Built-in Types, <https://docs.python.org/3.6/library/stdtypes.html#text-sequence-type-str>
2. Java™ Platform, Standard Edition 7 API Specification. Class String, <https://docs.oracle.com/javase/7/docs/api/java/lang/String.html>
3. Qt Documentation. QString Class. <http://doc.qt.io/qt-5/qstring.html>
4. string_view: a non-owning reference to a string, revision 3, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3609.html>
5. Integrating std::string_view and std::string, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0254r1.pdf>